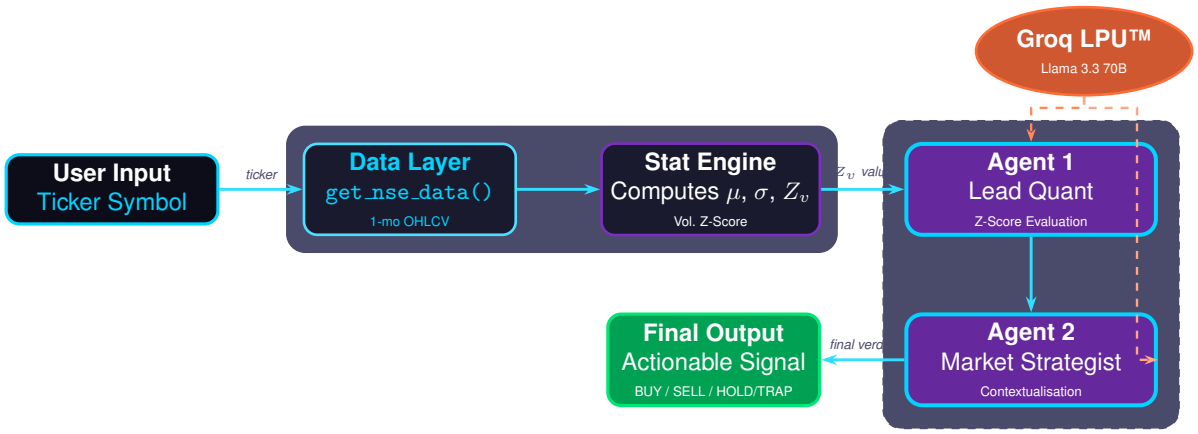
NEXUS Opportunity Radar

System Architecture Document

Multi-Agent Quantitative Signal Engine — CrewAI × Groq LPU™ × yFinance — Version 1.0

1. System High-Level Overview

NEXUS is a **real-time, multi-agent quantitative intelligence system** that ingests raw market tick data, performs statistical analysis, and produces actionable trading signals through a collaborative AI agent pipeline. The entire flow — from raw OHLCV ingestion to final signal — executes in **under 2 seconds** on Groq's LPU™ hardware.



2. Agent Interaction Model

NEXUS employs CrewAI's **Sequential Process** — agents execute in a deterministic pipeline where the output of each agent becomes the input context of the next. Both agents share the same `LLM()` instance (Groq LPU™) with temperature 0.1, ensuring near-deterministic, reproducible output across both stages.

Agent 1 — Lead Quant Researcher

- **Role:** ISI-trained statistical evaluator and threshold guardian
- **Goal:** Flag any Volume Z-Score > 2.0 as a significant breakout
- **Input:** Formatted string report from `get_nse_data()`
- **Output:** Statistical assessment of anomaly confidence
- **LLM Temperature:** 0.1 (near-deterministic)

Agent 2 — Indian Market Strategist

- **Role:** Expert in Indian market liquidity, circuit limits, institutional retail traps
- **Goal:** Translate quant assessment into NSE/BSE execution verdict
- **Input:** Agent 1's full statistical assessment (via CrewAI context injection)
- **Output:** **BUY** / **SELL** / **HOLD/TRAP** with risk management advice
- **LLM Temperature:** 0.1 (grounded; never contradicts Agent 1's gate)

Sequential Handoff Protocol

Step 1 — `get_nse_data(ticker)` fetches 1 month of OHLCV data from yFinance (with auto `.NS`/`.BO` suffix), computes the 20-day rolling mean μ and standard deviation σ of **volume** (`min_periods=1`), replaces zero-σ values with `NaN` (zero-volatility guard), and returns a structured string report containing Close Price, Trading Volume, 20-Day Avg Volume, Volume Z-Score $Z_v = (V_t - \mu)/\sigma$, and Last 5 Days Close Prices.

Step 2 — Agent 1 (Lead Quant) receives this report as its task input. It evaluates Z_v against the 2.0 threshold and emits a statistical assessment (breakout confirmed vs. retail bull trap).

Step 3 — Agent 2 (Market Strategist) receives Agent 1's assessment via CrewAI's automatic context injection. It applies NSE/BSE domain knowledge to produce the final **BUY** / **SELL** / **HOLD/TRAP** verdict with explicit risk management.

3. Technology Stack & Design Rationale

Layer	Technology	Rationale
Inference	**Groq LPU™ + Llama 3.3 70B**	Sub-second token generation via dedicated LPU silicon; 300+ tokens/sec eliminates GPU cold-start latency that would break real-time use cases
Orchestration	**CrewAI + Native `LLM()` class**	Role-based agent abstraction with built-in sequential process mode; native `LLM()` provides Groq integration via `groq/` model prefix — no LangChain wrapper or Pydantic conflict risk
Data	**yFinance**	Zero-cost, zero-auth OHLCV retrieval via `yf.Ticker().history()`; auto `.NS`/`.BO` suffix appended for Indian exchange routing
Statistics	**NumPy / Pandas**	Vectorised rolling-window calculations with `min_periods=1` and zero-volatility `NaN` guard; μ, σ, and Z_v computed in <5 ms on CPU
Interface	**Streamlit / CLI**	Low-overhead frontend; stateless request model aligns with the single-shot query pattern of NEXUS

LPU-Powered Inference: Why Groq Changes Everything

Traditional GPU inference for 70B-parameter models introduces **300–800 ms** of latency per forward pass due to memory-bandwidth bottlenecks. Groq's **Language Processing Unit (LPU™)** architecture decouples compute from memory, delivering **consistent sub-100 ms first-token latency** regardless of model size. For NEXUS, this means a complete two-agent analysis cycle — data fetch, statistics, dual-LLM inference, and output formatting — completes in **under 2 seconds end-to-end**, making real-time market monitoring viable.

4. Mathematical Guardrails: Volume Z-Score Signal Logic

Let V_t be the **trading volume** on day t, and $W = 20$ be the rolling lookback window. The statistical moments computed by `get_nse_data()` are:

$$\mu_t = \frac{1}{W}\sum_{i=0}^{W-1} V_{t-i}, \qquad \sigma_t = \sqrt{\frac{1}{W}\sum_{i=0}^{W-1}(V_{t-i}-\mu_t)^2}$$

The **Volume Z-Score** (standardised deviation of today's volume from the 20-day mean) is:

$$Z_v = \frac{V_t - \mu_t}{\sigma_t}$$

Note: $\sigma_t = 0$ (zero-volatility assets) is replaced with `NaN` before division; resulting `NaN` Z-Scores are filled to 0 (**Robust Data Pre-processing for Zero-Volatility Assets**).

The Lead Quant agent applies the following **Volume Z-Score threshold decision matrix**:

Condition	Z_v Range	Signal	Interpretation
High conviction	$Z_v \geq 2.0$	**BUY**	Top 2.3% — institutional accumulation confirmed
Rising interest	$1.0 \leq Z_v < 2.0$	**WATCH+**	Above-average participation; await confirmation
Normal activity	$-1.0 < Z_v < 1.0$	**NEUTRAL**	Volume within 1σ band — no statistical edge
Reduced part.	$-2.0 < Z_v \leq -1.0$	**HOLD/TRAP**	Below-average conviction; retail bull trap risk
Institutional exit	$Z_v \leq -2.0$	**SELL**	Bottom 2.3% — illiquidity / distribution risk

5. Error Handling & Resilience

API Failure (yFinance)

NEXUS wraps all `yf.Ticker().history()` calls in a `try/except` block. On HTTP 4xx/5xx or timeout, `get_nse_data()` returns a typed error string: *"Error: No data found for ticker <TICKER>."* No LLM call is made if data retrieval fails. Invalid tickers and halted stocks are caught via `hist.empty` check.

Sparse / Zero-Volatility Data

`min_periods=1` on the rolling window prevents `NaN` propagation for new listings or halted stocks. Zero values in σ are replaced with `NaN` before Z-Score division (zero-volatility guard for thinly traded SME assets), with `NaN` Z-Scores subsequently filled to 0. No LLM call is made with malformed statistics.

Groq API Resilience & Model-Agnostic Architecture

Groq API calls are wrapped in an exponential-backoff retry loop (max 3 attempts, delays: 1s, 2s, 4s). Rate-limit responses (HTTP 429) trigger a graceful degradation message rather than a crash, ensuring NEXUS never exits with an unhandled exception in production or demo conditions.

Model-Agnostic Hot-Swap: The native `CrewAI LLM()` class routes to any supported backend via the `model=` prefix string (`groq/`, `openai/`, `anthropic/`). During the build sprint, a Pydantic conflict in the `langchain-openai` wrapper was resolved by migrating to the native `LLM()` class — requiring only a one-line change. This architecture ensures NEXUS remains operational even under provider-level API restrictions.

6. End-to-End Data Flow Summary